java.nio library (API at http://mng.bz/uojM). As a start, try implementing an asynchronous read from an AsynchronousFileChannel (API at http://mng.bz/X30L).[17]

```
def read(file: AsynchronousFileChannel,
         fromPosition: Long,
         numBytes: Int): Par[Either[Throwable, Array[Byte]]]
```

13.6 A general-purpose IO type

We can now formulate a general methodology of writing programs that perform I/O. For any given set of I/O operations that we want to support, we can write an algebraic data type whose case classes represent the individual operations. For example, we could have a Files data type for file I/O, a DB data type for database access, and use something like Console for interacting with standard input and output. For any such data type F, we can generate a free monad Free[F,A] in which to write our programs. These can be tested individually and then finally "compiled" down to a lower-level I/O type, what we earlier called Async:

```
type IO[A] = Free[Par, A]
```

This IO type supports both trampolined sequential execution (because of Free) and asynchronous execution (because of Par). In our main program, we bring all of the individual effect types together under this most general type. All we need is a translation from any given F to Par.

13.6.1 The main program at the end of the universe

When the JVM calls into our main program, it expects a main method with a specific signature. The return type of this method is Unit, meaning that it's expected to have some side effects. But we can delegate to a pureMain program that's entirely pure! The only thing the main method does in that case is interpret our pure program, actually performing the effects.

Listing 13.8 Turning side effects into just effects

Interprets the IO action and actually performs the effect by turning IO[A] into Par[A] and then A. The name of this method reflects that it's unsafe to call (because it has side effects).

All that the main method does is interpret our pureMain.

```
abstract class App {
  import java.util.concurrent._

  def unsafePerformIO[A](a: IO[A])(pool: ExecutorService): A =
    Par.run(pool)(run(a)(parMonad))

  def main(args: Array[String]): Unit = {
```

[17] This requires Java 7 or better.